PyTorch & TensorFlow Complete Cheat Sheet

Table of Contents

Installation & Setup

PyTorch

```bash
pip install torch torchvision torchaudio
```

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision
import torchvision.transforms as transforms

# Check CUDA availability
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

TensorFlow

```bash
pip install tensorflow
```

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
import numpy as np

# Check GPU availability
print("GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
```

Tensor Operations

Creating Tensors

PyTorch:

```python
# From data
x = torch.tensor([1, 2, 3])
x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)

# Zeros, ones, random
zeros = torch.zeros(3, 4)
ones = torch.ones(2, 3)
rand = torch.rand(3, 3)  # Uniform [0, 1)
randn = torch.randn(3, 3)  # Normal distribution
empty = torch.empty(2, 2)

# From numpy
import numpy as np
np_array = np.array([1, 2, 3])
tensor = torch.from_numpy(np_array)

# Like another tensor
x_like = torch.zeros_like(x)
x_like = torch.ones_like(x)
```

TensorFlow:

```python
```

```
# From data
x = tf.constant([1, 2, 3])
x = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)

# Zeros, ones, random
zeros = tf.zeros([3, 4])
ones = tf.ones([2, 3])
rand = tf.random.uniform([3, 3])  # Uniform [0, 1)
randn = tf.random.normal([3, 3])  # Normal distribution

# From numpy
np_array = np.array([1, 2, 3])
tensor = tf.constant(np_array)

# Like another tensor
x_like = tf.zeros_like(x)
x_like = tf.ones_like(x)
```

Tensor Attributes & Operations

PyTorch:

python

```
x = torch.randn(3, 4)

# Attributes
print(x.shape)  # torch.Size([3, 4])
print(x.dtype)  # torch.float32
print(x.device)  # cpu or cuda

# Move to device
x = x.to(device)
x = x.cuda()  # Move to GPU
x = x.cpu()   # Move to CPU

# Reshape
x = x.view(12)  # Must be contiguous
x = x.reshape(2, 6)  # More flexible
x = x.flatten()

# Transpose
x = x.t()  # 2D transpose
x = x.transpose(0, 1)  # Swap dimensions
x = x.permute(1, 0, 2)  # Rearrange dimensions

# Type conversion
x = x.float()
x = x.long()
x = x.int()
```

TensorFlow:

```
python
```

```
x = tf.random.normal([3, 4])

# Attributes
print(x.shape)  # (3, 4)
print(x.dtype)  # float32

# Reshape
x = tf.reshape(x, [12])
x = tf.reshape(x, [2, 6])
x = tf.reshape(x, [-1])  # Flatten

# Transpose
x = tf.transpose(x)
x = tf.transpose(x, perm=[1, 0, 2])

# Type conversion
x = tf.cast(x, tf.float32)
x = tf.cast(x, tf.int32)
```

Mathematical Operations

PyTorch:

python

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

# Element-wise operations
c = a + b
c = a - b
c = a * b
c = a / b
c = a ** 2
c = torch.sqrt(a)
c = torch.exp(a)
c = torch.log(a)

# Matrix operations
A = torch.randn(3, 4)
B = torch.randn(4, 5)
C = torch.mm(A, B)  # Matrix multiplication
C = A @ B  # Same as above

# Batch matrix multiplication
A = torch.randn(10, 3, 4)
B = torch.randn(10, 4, 5)
C = torch.bmm(A, B)  # (10, 3, 5)

# Reduction operations
mean = a.mean()
sum_val = a.sum()
max_val = a.max()
min_val = a.min()
argmax = a.argmax()
```

TensorFlow:

```
python
```

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

# Element-wise operations
c = a + b
c = a - b
c = a * b
c = a / b
c = tf.pow(a, 2)
c = tf.sqrt(tf.cast(a, tf.float32))
c = tf.exp(tf.cast(a, tf.float32))
c = tf.math.log(tf.cast(a, tf.float32))

# Matrix operations
A = tf.random.normal([3, 4])
B = tf.random.normal([4, 5])
C = tf.matmul(A, B)
C = A @ B  # Same as above

# Reduction operations
mean = tf.reduce_mean(a)
sum_val = tf.reduce_sum(a)
max_val = tf.reduce_max(a)
min_val = tf.reduce_min(a)
argmax = tf.argmax(a)
```

Indexing & Slicing

PyTorch:

```
python

x = torch.randn(4, 5)

# Basic indexing
first_row = x[0]
first_col = x[:, 0]
subset = x[1:3, 2:4]

# Boolean indexing
mask = x > 0
positive = x[mask]

# Advanced indexing
indices = torch.tensor([0, 2])
selected = x[indices]
```

TensorFlow:

```python
x = tf.random.normal([4, 5])

# Basic indexing
first_row = x[0]
first_col = x[:, 0]
subset = x[1:3, 2:4]

# Boolean indexing
mask = x > 0
positive = tf.boolean_mask(x, mask)

# Advanced indexing
indices = [0, 2]
selected = tf.gather(x, indices)
```

Neural Network Layers

Common Layers

PyTorch:

```python
```

```
import torch.nn as nn

# Linear (Dense) layer
linear = nn.Linear(in_features=10, out_features=5)

# Convolutional layers
conv1d = nn.Conv1d(in_channels=3, out_channels=16, kernel_size=3)
conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
conv3d = nn.Conv3d(in_channels=3, out_channels=16, kernel_size=3)

# Pooling layers
maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
avgpool = nn.AvgPool2d(kernel_size=2)
adaptivepool = nn.AdaptiveAvgPool2d((1, 1))

# Normalization layers
batchnorm = nn.BatchNorm2d(num_features=16)
layernorm = nn.LayerNorm(normalized_shape=10)

# Dropout
dropout = nn.Dropout(p=0.5)
dropout2d = nn.Dropout2d(p=0.5)

# Activation functions
relu = nn.ReLU()
sigmoid = nn.Sigmoid()
tanh = nn.Tanh()
leaky_relu = nn.LeakyReLU(0.01)
softmax = nn.Softmax(dim=1)

# Recurrent layers
rnn = nn.RNN(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
lstm = nn.LSTM(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
gru = nn.GRU(input_size=10, hidden_size=20, num_layers=2, batch_first=True)

# Embedding
embedding = nn.Embedding(num_embeddings=1000, embedding_dim=128)
```

TensorFlow/Keras:

```
python
```

```
from tensorflow.keras import layers

# Dense layer
dense = layers.Dense(units=5, activation='relu')

# Convolutional layers
conv1d = layers.Conv1D(filters=16, kernel_size=3, activation='relu')
conv2d = layers.Conv2D(filters=16, kernel_size=3, strides=1, padding='same', activation='relu')
conv3d = layers.Conv3D(filters=16, kernel_size=3, activation='relu')

# Pooling layers
maxpool = layers.MaxPooling2D(pool_size=2, strides=2)
avgpool = layers.AveragePooling2D(pool_size=2)
globalavgpool = layers.GlobalAveragePooling2D()

# Normalization layers
batchnorm = layers.BatchNormalization()
layernorm = layers.LayerNormalization()

# Dropout
dropout = layers.Dropout(rate=0.5)

# Activation functions (as layers)
relu = layers.ReLU()
# Or use activation parameter in layers
dense_with_activation = layers.Dense(10, activation='relu')

# Recurrent layers
simple_rnn = layers.SimpleRNN(units=20, return_sequences=True)
lstm = layers.LSTM(units=20, return_sequences=True)
gru = layers.GRU(units=20, return_sequences=True)

# Embedding
embedding = layers.Embedding(input_dim=1000, output_dim=128)

# Flatten
flatten = layers.Flatten()
```

Model Building

Sequential Models

PyTorch:

```python
# Method 1: Using nn.Sequential
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)

# Method 2: Using OrderedDict
from collections import OrderedDict
model = nn.Sequential(OrderedDict([
    ('fc1', nn.Linear(784, 256)),
    ('relu1', nn.ReLU()),
    ('dropout1', nn.Dropout(0.2)),
    ('fc2', nn.Linear(256, 10))
]))
```

TensorFlow/Keras:

```python
# Method 1: Sequential API
model = keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

# Method 2: Adding layers one by one
model = keras.Sequential()
model.add(layers.Dense(256, activation='relu', input_shape=(784,)))
model.add(layers.Dropout(0.2))
model.add(layers.Dense(10, activation='softmax'))
```

Custom Models

PyTorch:

```python
```

```
class CustomModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(CustomModel, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = CustomModel(784, 256, 10).to(device)
```

TensorFlow/Keras:

```
python

class CustomModel(keras.Model):
    def __init__(self, hidden_size, num_classes):
        super(CustomModel, self).__init__()
        self.fc1 = layers.Dense(hidden_size, activation='relu')
        self.dropout = layers.Dropout(0.2)
        self.fc2 = layers.Dense(num_classes, activation='softmax')

    def call(self, inputs, training=False):
        x = self.fc1(inputs)
        x = self.dropout(x, training=training)
        x = self.fc2(x)
        return x

model = CustomModel(256, 10)
```

CNN Example

PyTorch:

```
python
```

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = x.view(-1, 64 * 8 * 8)  # Flatten
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = CNN().to(device)
```

TensorFlow/Keras:

```
python

model = keras.Sequential([
    layers.Conv2D(32, 3, padding='same', activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

RNN/LSTM Example

PyTorch:

```
python
```

```
class RNNModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes):
        super(RNNModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)

        out, _ = self.lstm(x, (h0, c0))
        out = self.fc(out[:, -1, :])  # Last time step
        return out

model = RNNModel(input_size=28, hidden_size=128, num_layers=2, num_classes=10).to(device)
```

TensorFlow/Keras:

```
python

model = keras.Sequential([
    layers.LSTM(128, return_sequences=True, input_shape=(None, 28)),
    layers.LSTM(128),
    layers.Dense(10, activation='softmax')
])
```

Training Loop

Basic Training Loop

PyTorch:

```
python
```

```
# Setup
model = CustomModel(784, 256, 10).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
num_epochs = 10
for epoch in range(num_epochs):
    model.train()  # Set to training mode
    running_loss = 0.0

    for i, (inputs, labels) in enumerate(train_loader):
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Zero the gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}')

# Evaluation
model.eval()  # Set to evaluation mode
with torch.no_grad():
    correct = 0
    total = 0
    for inputs, labels in test_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
    accuracy = 100 * correct / total
    print(f'Test Accuracy: {accuracy:.2f}%')
```

TensorFlow/Keras:

```
python

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Training
history = model.fit(
    train_dataset,
    epochs=10,
    validation_data=test_dataset,
    verbose=1
)

# Evaluation
test_loss, test_acc = model.evaluate(test_dataset)
print(f'Test Accuracy: {test_acc:.4f}')

# Custom training loop (advanced)
optimizer = keras.optimizers.Adam()
loss_fn = keras.losses.SparseCategoricalCrossentropy()

@tf.function
def train_step(x, y):
    with tf.GradientTape() as tape:
        predictions = model(x, training=True)
        loss = loss_fn(y, predictions)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    return loss

for epoch in range(10):
    for x_batch, y_batch in train_dataset:
        loss = train_step(x_batch, y_batch)
    print(f'Epoch {epoch+1}, Loss: {loss.numpy():.4f}')
```

Loss Functions & Optimizers

Loss Functions

PyTorch:

```python
import torch.nn as nn

# Classification
criterion = nn.CrossEntropyLoss()  # For multi-class
criterion = nn.BCELoss()  # Binary cross-entropy
criterion = nn.BCEWithLogitsLoss()  # BCE with logits (more stable)
criterion = nn.NLLLoss()  # Negative log likelihood

# Regression
criterion = nn.MSELoss()  # Mean squared error
criterion = nn.L1Loss()  # Mean absolute error
criterion = nn.SmoothL1Loss()  # Huber loss

# Custom weights
weights = torch.tensor([1.0, 2.0, 3.0])
criterion = nn.CrossEntropyLoss(weight=weights)
```

TensorFlow/Keras:

```python
```

```
from tensorflow.keras import losses

# Classification
loss = losses.SparseCategoricalCrossentropy()
loss = losses.CategoricalCrossentropy()
loss = losses.BinaryCrossentropy()

# Regression
loss = losses.MeanSquaredError()
loss = losses.MeanAbsoluteError()
loss = losses.Huber()

# In model.compile
model.compile(
    loss='sparse_categorical_crossentropy',
    # or
    loss=losses.SparseCategoricalCrossentropy()
)
```

Optimizers

PyTorch:

```
python
```

```
import torch.optim as optim

# Basic optimizers
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)

# Different learning rates for different layers
optimizer = optim.Adam([
    {'params': model.conv_layers.parameters(), 'lr': 1e-4},
    {'params': model.fc_layers.parameters(), 'lr': 1e-3}
])

# Learning rate scheduler
from torch.optim.lr_scheduler import StepLR, ReduceLROnPlateau

scheduler = StepLR(optimizer, step_size=30, gamma=0.1)
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=10)

# Usage in training loop
for epoch in range(num_epochs):
    # Training code...
    scheduler.step()  # Update learning rate
```

TensorFlow/Keras:

```
python
```

```
from tensorflow.keras import optimizers

# Basic optimizers
optimizer = optimizers.SGD(learning_rate=0.01, momentum=0.9)
optimizer = optimizers.Adam(learning_rate=0.001)
optimizer = optimizers.RMSprop(learning_rate=0.01)

# Learning rate schedules
lr_schedule = keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    decay_rate=0.9
)
optimizer = optimizers.Adam(learning_rate=lr_schedule)

# Callbacks for learning rate
from tensorflow.keras.callbacks import ReduceLROnPlateau

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.1,
    patience=10,
    min_lr=1e-7
)

model.fit(train_dataset, epochs=10, callbacks=[reduce_lr])
```

Data Loading

Custom Dataset

PyTorch:

```
python
```

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    def __init__(self, data, labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        label = self.labels[idx]

        if self.transform:
            sample = self.transform(sample)

        return sample, label

# Create dataset and dataloader
dataset = CustomDataset(data, labels)
dataloader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=4,
    pin_memory=True  # For GPU
)

# Usage
for batch_data, batch_labels in dataloader:
    # Training code
    pass
```

TensorFlow:

```
python
```

```
# From numpy arrays
train_dataset = tf.data.Dataset.from_tensor_slices((train_data, train_labels))
train_dataset = train_dataset.shuffle(10000).batch(32).prefetch(tf.data.AUTOTUNE)

# From generator
def data_generator():
    for i in range(len(data)):
        yield data[i], labels[i]

dataset = tf.data.Dataset.from_generator(
    data_generator,
    output_signature=(
        tf.TensorSpec(shape=(28, 28), dtype=tf.float32),
        tf.TensorSpec(shape=(), dtype=tf.int32)
    )
)
dataset = dataset.batch(32).prefetch(tf.data.AUTOTUNE)
```

Image Data Augmentation

PyTorch:

```
python

from torchvision import transforms

# Define transforms
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# Apply to dataset
from torchvision.datasets import ImageFolder

dataset = ImageFolder(root='path/to/data', transform=transform)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

TensorFlow:

```
python
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    zoom_range=0.2
)

# From directory
train_generator = datagen.flow_from_directory(
    'path/to/data',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

# Using tf.data
def augment(image, label):
    image = tf.image.resize(image, [224, 224])
    image = tf.image.random_flip_left_right(image)
    image = tf.image.random_brightness(image, 0.2)
    return image, label

dataset = dataset.map(augment, num_parallel_calls=tf.data.AUTOTUNE)
```

Model Saving & Loading

PyTorch:

```
python
```

```
# Save entire model
torch.save(model, 'model.pth')

# Load entire model
model = torch.load('model.pth')

# Save only state dict (recommended)
torch.save(model.state_dict(), 'model_weights.pth')

# Load state dict
model = CustomModel(784, 256, 10)
model.load_state_dict(torch.load('model_weights.pth'))
model.eval()

# Save checkpoint (with optimizer state)
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss
}
torch.save(checkpoint, 'checkpoint.pth')

# Load checkpoint
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
epoch = checkpoint['epoch']
loss = checkpoint['loss']
```

TensorFlow/Keras:

```
python
```

```
# Save entire model (architecture + weights + optimizer)
model.save('my_model.h5')
model.save('my_model')  # SavedModel format (recommended)

# Load model
model = keras.models.load_model('my_model.h5')

# Save only weights
model.save_weights('model_weights.h5')

# Load weights
model.load_weights('model_weights.h5')

# Save with checkpoints during training
checkpoint_callback = keras.callbacks.ModelCheckpoint(
    filepath='checkpoint.h5',
    save_weights_only=True,
    save_best_only=True,
    monitor='val_loss',
    mode='min'
)

model.fit(train_dataset, epochs=10, callbacks=[checkpoint_callback])
```

Common Patterns

Transfer Learning

PyTorch:

```
python
```

```
import torchvision.models as models

# Load pretrained model
model = models.resnet50(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace final layer
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)

# Only train final layer
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)
```

TensorFlow/Keras:

```
python

# Load pretrained model
base_model = keras.applications.ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

# Freeze base model
base_model.trainable = False

# Add custom layers
model = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Mixed Precision Training

PyTorch:

```
python
```

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for inputs, labels in train_loader:
    optimizer.zero_grad()

    with autocast():
        outputs = model(inputs)
        loss = criterion(outputs, labels)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

TensorFlow:

```
python

from tensorflow.keras import mixed_precision

# Enable mixed precision
mixed_precision.set_global_policy('mixed_float16')

# Model automatically uses mixed precision
model = keras.Sequential([...])
model.compile(optimizer='adam', loss='categorical_crossentropy')
```

Gradient Clipping

PyTorch:

```
python

# Clip by norm
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Clip by value
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=0.5)

# In training loop
optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

TensorFlow:

python

```
# In optimizer
optimizer = keras.optimizers.Adam(clipnorm=1.0)  # Clip by norm
optimizer = keras.optimizers.Adam(clip
```